



CMC UNIVERSITY

Aspire to Inspire the Digital World

AUTHENTICATION & AUTHORIZATION

Ngô Việt Anh

Khoa CNTT&TT, Trường Đại học CMC

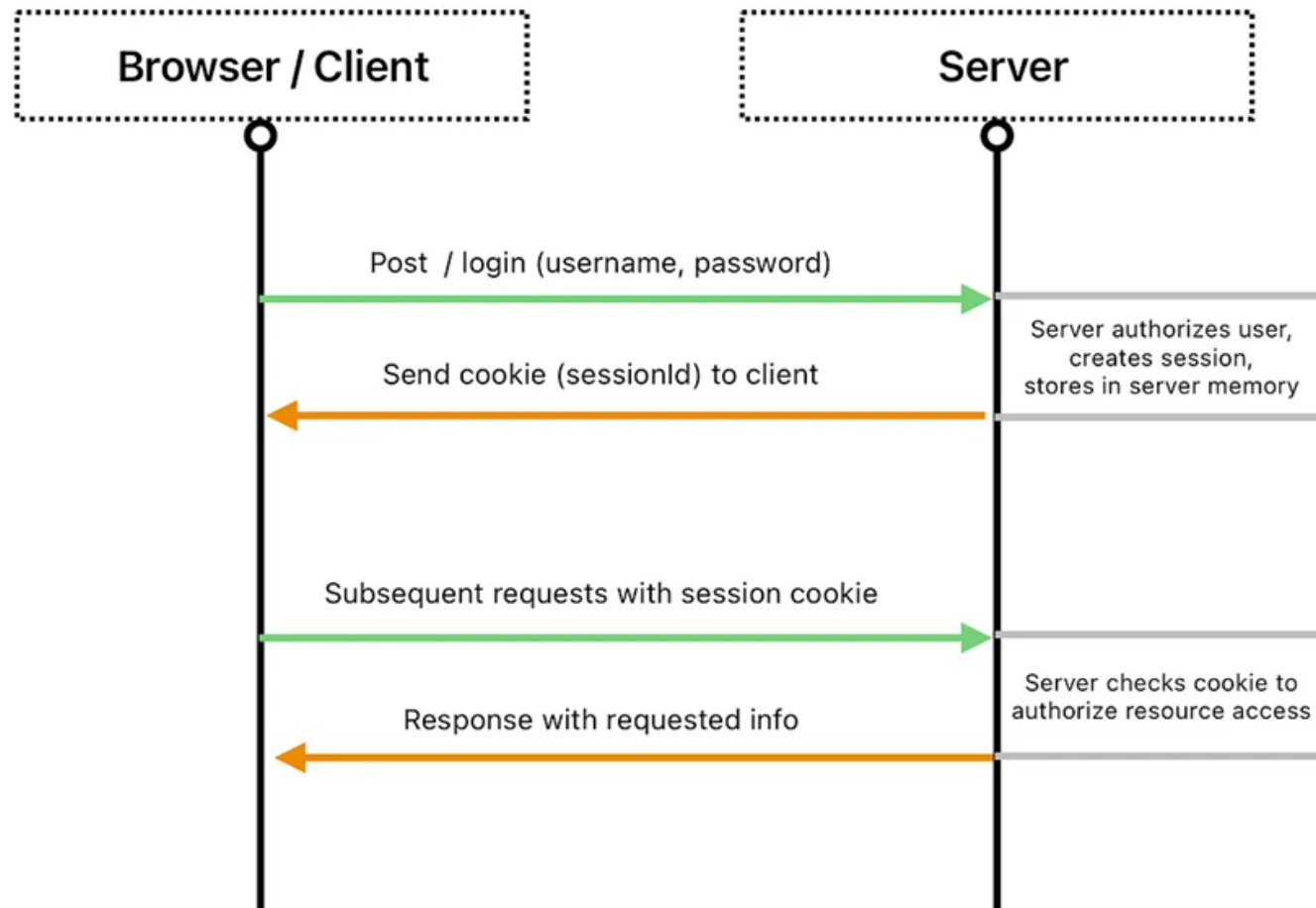
Hà Nội, tháng 12 năm 2025

- Trong ASP.NET Core có nhiều phương thức xác thực (authentication) để phục vụ cho các loại ứng dụng khác nhau, từ ứng dụng web truyền thống, API đến SPA (Single Page Application) hay mobile apps
 - Cookie Authentication
 - Windows Authentication
 - JWT (JSON Web Token) Bearer Authentication
 - External Authentication (OAuth 2.0 / OpenID Connect)
 - Custom Authentication Schemes

Cookie Authentication

- Cookie là một đoạn dữ liệu nhỏ được lưu trữ trên máy tính của người dùng (client-side) bởi trình duyệt web. Cookie thường được sử dụng để lưu trữ thông tin về người dùng, chẳng hạn như thông tin đăng nhập, tùy chọn cá nhân, hoặc các dữ liệu khác mà máy chủ (server) cần ghi nhớ giữa các lần truy cập.
- Cookie được gửi từ máy chủ đến trình duyệt và được lưu trữ trên máy tính của người dùng. Khi người dùng truy cập lại trang web, trình duyệt sẽ gửi cookie đó trở lại máy chủ, giúp máy chủ nhận diện người dùng và cung cấp trải nghiệm cá nhân hóa.

- **Cookie Authentication** là phương thức thường dùng cho các ứng dụng web (MVC, Razor Pages) nơi thông tin đăng nhập của người dùng được lưu dưới dạng cookie.
- Khi người dùng đăng nhập thành công, server sẽ tạo một ticket chứa thông tin xác thực, mã hóa và lưu vào cookie.
- Mỗi lần người dùng gửi yêu cầu, cookie được gửi kèm theo để server xác nhận danh tính.



```
// Thêm dịch vụ Authentication với Cookie
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Account/Login"; // Đường dẫn đến trang đăng nhập
        options.ExpireTimeSpan = TimeSpan.FromMinutes(30);
    });

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.MapDefaultControllerRoute();
```


- **Ưu điểm:**

- Thực hiện đơn giản, phù hợp với ứng dụng web truyền thống.
- Dễ tích hợp với ASP.NET Core Identity.

- **Nhược điểm:**

- Bảo mật: Dễ bị tấn công CSRF và đánh cắp cookie.
- Khả năng mở rộng: Khó khăn trong hệ thống phân tán.
- Hiệu suất: Tăng tải cho máy chủ do kích thước yêu cầu lớn.
- Quản lý phiên: Khó quản lý phiên trên nhiều thiết bị.
- Phụ thuộc vào trình duyệt: Không phù hợp với các ứng dụng không dựa trên trình duyệt.
- Thời gian hết hạn: Khó cân bằng giữa trải nghiệm người dùng và bảo mật.
- Tuân thủ quy định: Cần tuân thủ các quy định về quyền riêng tư.
- Chia sẻ tài nguyên: Khó chia sẻ cookie giữa các domain khác nhau.

Thực hành - Cookie Authentication

Viết 1 ứng dụng ASP.NET CORE MVC demo việc sử dụng Cookie để xác thực

- Thường dùng trong các ứng dụng nội bộ (Intranet) khi người dùng đăng nhập bằng tài khoản Windows.
- ASP.NET Core hỗ trợ Windows Authentication khi ứng dụng được host trên IIS hoặc chạy trên môi trường Windows.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddAuthentication(IISDefaults.AuthenticationScheme);

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

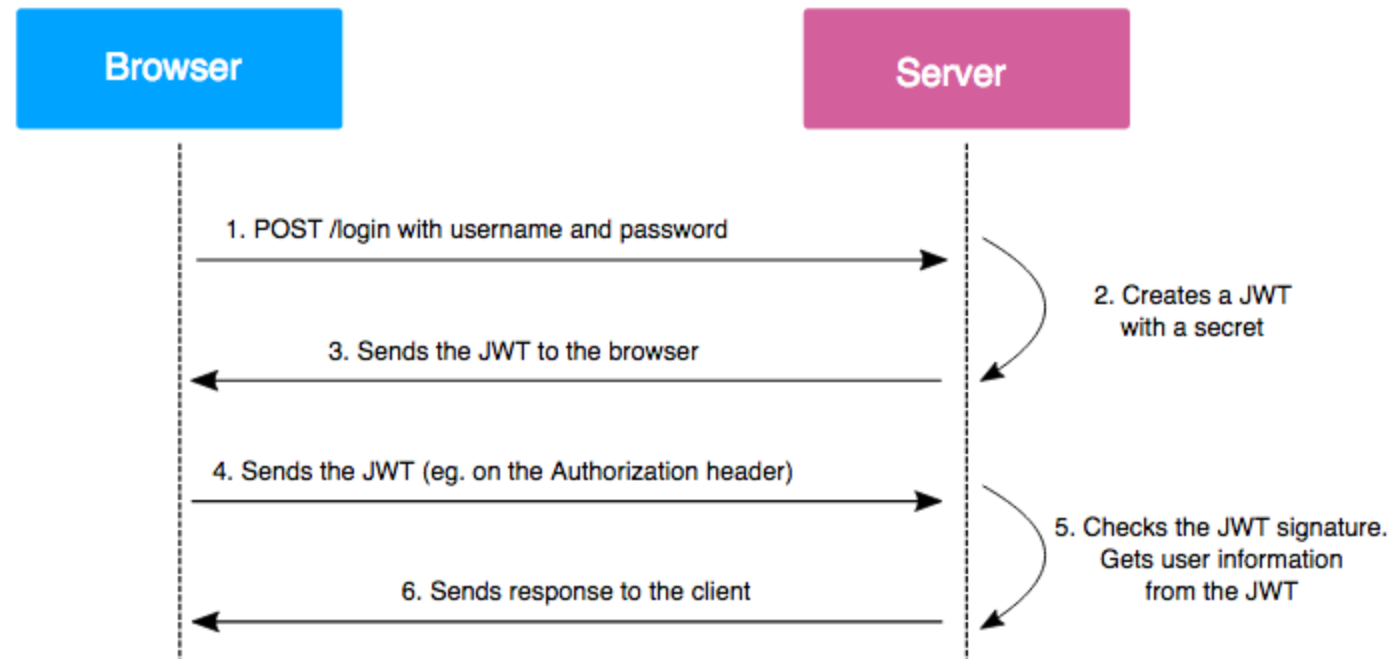
app.MapDefaultControllerRoute();

app.Run();
```

- **Ưu điểm:**
 - Tích hợp sẵn với Active Directory, không cần quản lý mật khẩu riêng.
 - Phù hợp cho môi trường doanh nghiệp, nội bộ.
- **Nhược điểm:**
 - Không thích hợp cho ứng dụng public trên Internet.
 - Yêu cầu môi trường Windows và cấu hình trên IIS.

JWT Authentication

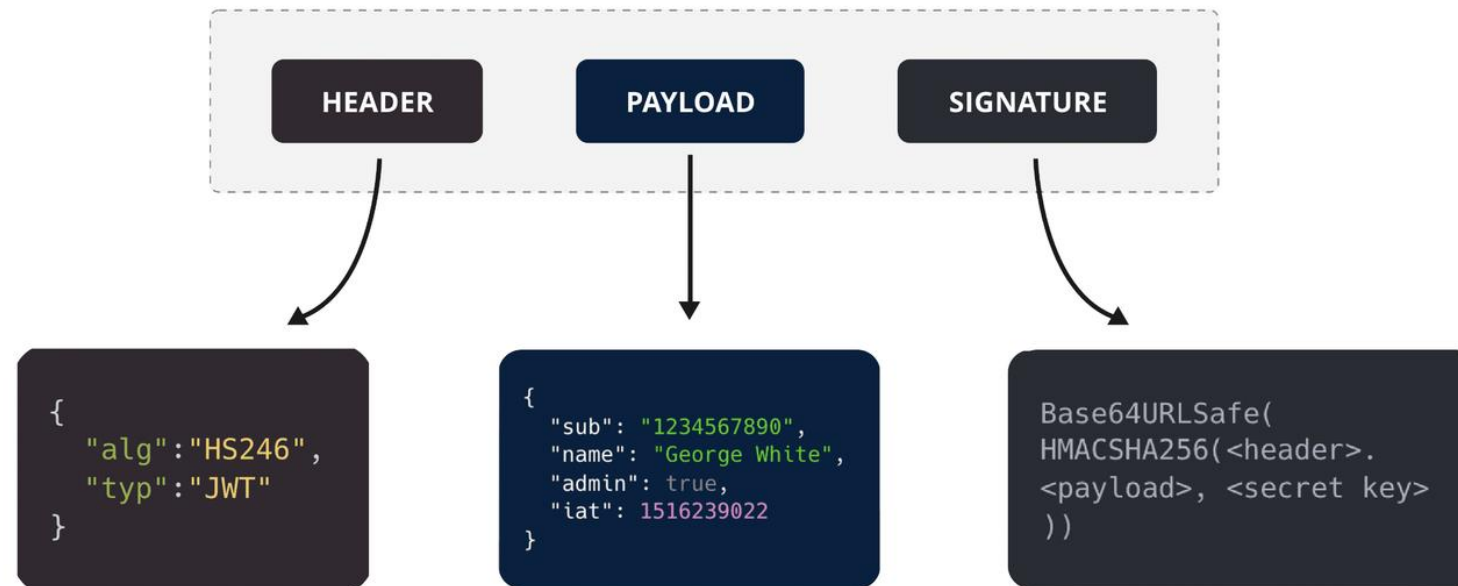
- Phương thức này thường dùng cho các API, SPA hoặc mobile apps.
- Sau khi người dùng đăng nhập thành công, server phát hành một **JWT token** chứa các claim (danh tính, quyền hạn...) cho người dùng.
- Client lưu trữ token (thường trong LocalStorage hoặc SessionStorage) và gửi kèm trong header của mỗi yêu cầu (thường là **Authorization: Bearer {token}**).



- **JWT** là một chuẩn token-based authentication, giúp chuyển thông tin xác thực dưới dạng một chuỗi có cấu trúc gồm 3 phần:
- **Header:** Chứa thông tin thuật toán ký (ví dụ: HMAC SHA256) và kiểu token.
- **Payload:** Chứa các **claims** (những thông tin về người dùng, quyền hạn, thời gian hết hạn, ...).
- **Signature:** Được tạo bằng cách mã hóa header và payload kèm theo một khóa bí mật (secret key) nhằm đảm bảo tính toàn vẹn của token.
- Khi người dùng đăng nhập thành công, server tạo ra một token JWT, gửi về cho client. Sau đó, client sẽ gửi token này trong header của mỗi yêu cầu (thường dùng `Authorization: Bearer {token}`) để xác thực.

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

Structure of a JSON Web Token (JWT)



- Header thông thường sẽ bao gồm 2 thành phần, gồm : Kiểu token, mà với trường hợp này luôn luôn là JWT, và thuật toán mã hoá được sử dụng, ví dụ như HMAC hoặc RSA.

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

- *Payload* trong JWT chứa các *claims*.
- Trong lĩnh vực an toàn thông tin, *claims* đề cập đến sự tuyên bố quyền truy cập hoặc quyền sử dụng tài nguyên.
- *Claims* là tập hợp các thông tin đại diện cho một thực thể (*object*) (ví dụ : `user_id`) và một số thông tin đi kèm. *Claims* sẽ có dạng *Key - Value*. Do đó, chúng ta có thể hiểu rằng, *claims* ám chỉ việc yêu cầu truy xuất tài nguyên cho *object* tương ứng.
- Có 3 kiểu *claims*, bao gồm *registered*, *public*, and *private claims*.

- *Registered Claims* là các thành phần được xác định trước của *claims*. Thành phần này mặc dù không bắt buộc, nhưng là thành phần nên có để cung cấp một số chức năng và thông tin hữu ích.
- Một số *registered claims* bao gồm :
 - _{iss} (issuer): Tổ chức, đơn vị cung cấp, phát hành *JWT*.
 - _{sub} (subject): Chủ thể của *JWT*, xác định rằng đây là người sở hữu hoặc có quyền truy cập các *resource* (tài nguyên).
 - _{aud} (audience): Được hiểu là người nhận thông tin, và có thể xác thực tính hợp lệ của *JWT*.
 - _{exp} (expiration time): Thời hạn của *JWT*, vượt quá thời gian này, *JWT* được coi là không hợp lệ

- Public claims là các thành phần được xác định bởi người sử dụng JWT, được sử dụng rộng rãi trong JWT. Mặc dù việc sử dụng public claims không phải là bắt buộc, tuy nhiên để tránh xảy ra xung đột
- Một số *public claims* điển hình :
 - name, given_name, family_name, middle_name: Thông tin tên nói chung của user
 - email: Thông tin email của user.
 - locale : Địa chỉ của user.
 - profile, picture : Thông tin của trang web gửi đến.

Claim Name	Claim Description
iss	Issuer
sub	Subject
aud	Audience
exp	Expiration Time
nbf	Not Before
iat	Issued At
jti	JWT ID
name	Full name
given_name	Given name(s) or first name(s)
family_name	Surname(s) or last name(s)
middle_name	Middle name(s)
nickname	Casual name
preferred_username	Shorthand name by which the End-User wishes to be referred to
profile	Profile page URL
picture	Profile picture URL

Danh sách các [public claims](#)

- Các bên sử dụng JWT có thể sẽ cần sử dụng đến claims không phải là Registered Claims, cũng không được định nghĩa trước như Public Claims. Đây là phần thông tin mà các bên tự thoả thuận với nhau, không có tài liệu hay tiêu chuẩn nào dành cho private claims.

- *Signature* là phần cuối cùng của *JWT*, có chức năng xác thực danh tính người gửi. Để tạo ra một *signature* chính xác, ta cần *encode* phần *header*, phần *payload*, chọn *cryptography* (mật mã khoá) thích hợp đã được xác định ở *header* kèm *secret key* và thực hiện *sign*.
- Ví dụ, nếu chúng ta lựa chọn hàm *HMACSHA256*, *function* thực hiện *sign* sẽ có dạng như sau:

```
HMACSHA256(  
  base64UrlEncode(header) + "." +  
  base64UrlEncode(payload),  
  secret)
```

- Cả bên gửi và bên nhận JWT đều sẽ xử dụng hàm này để xác định phần *signature*, nếu thông tin này của cả 2 bên khác nhau, JWT này được coi là không hợp lệ.

```
// Cấu hình các thông số JWT trong appsettings.json hoặc trực tiếp tại đây
var jwtSettings = builder.Configuration.GetSection("Jwt");
var secretKey = jwtSettings["Key"];

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = jwtSettings["Issuer"],
        ValidAudience = jwtSettings["Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secretKey))
    };
});

builder.Services.AddAuthorization();
```

- Mục tiêu chính của đoạn code: Đăng ký và cấu hình xác thực cho ứng dụng sử dụng JWT Bearer. Khi một request được gửi đến, middleware sẽ:
 - Tìm token trong header (Authorization: Bearer {token}).
 - Sử dụng các tham số trong TokenValidationParameters để kiểm tra:
 - Người phát hành (Issuer) và đối tượng nhận (Audience) của token.
 - Thời gian sống của token (token chưa hết hạn).
 - Chữ ký số của token dựa trên khóa ký (secret key).
- Kết quả: Nếu token hợp lệ, middleware sẽ xác thực người dùng và gán các thông tin claims từ token vào HttpContext.User, cho phép các controller và action có thể truy cập thông tin người dùng đã xác thực.

- **Ưu điểm:**

- Phù hợp với kiến trúc microservices, API và các ứng dụng không phụ thuộc vào cookie.
- Không cần lưu trạng thái trên server (stateless).

- **Nhược điểm:**

- Cần có cơ chế làm mới token (Refresh Token) để duy trì phiên đăng nhập an toàn.
- Token có thời hạn sử dụng nên cần xử lý lỗi hết hạn token một cách hợp lý.

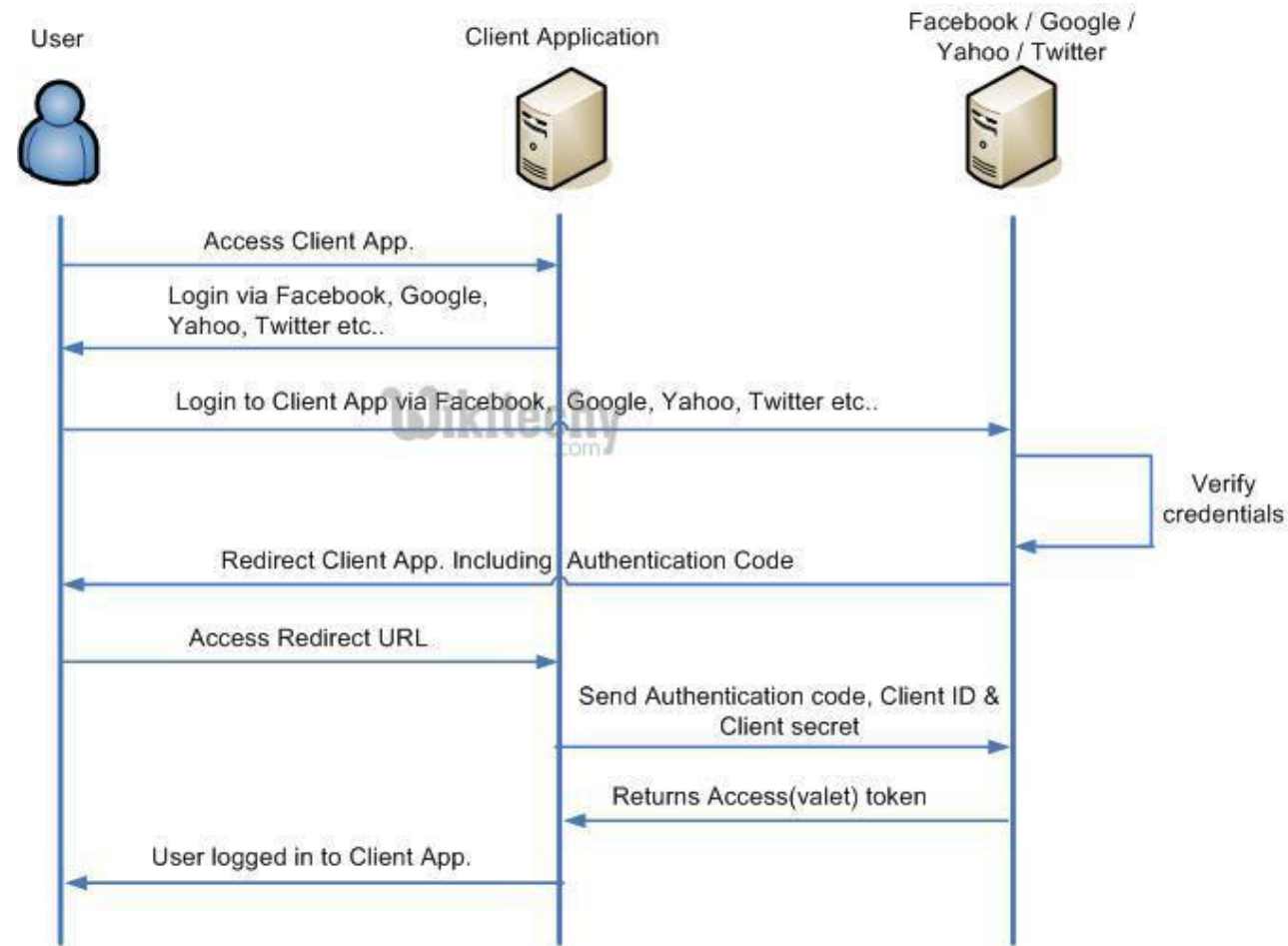


Viết 1 ứng dụng ASP.NET CORE MVC
hoặc ASP.NET CORE API demo việc sử dụng JWT
để xác thực.

Thực hành - JWT Authentication

External Authentication

- Cho phép ứng dụng của bạn tích hợp xác thực với các nhà cung cấp bên thứ ba như Google, Facebook, Microsoft, Twitter, ...
- Người dùng có thể đăng nhập bằng tài khoản của các dịch vụ này, không cần tạo tài khoản mới cho ứng dụng của bạn.



- Đăng ký tài khoản trên <https://cloud.google.com/>
- Chọn Console -> APIs & Service -> Credentials
- Tạo mới credential -> Chọn application type -> Add uri
- Lấy ClientID và Secret

```
builder.Services.AddAuthentication(options =>
{
    options.DefaultScheme = CookieAuthenticationDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = "Google";
})
.AddCookie()
.AddGoogle("Google", options =>
{
    options.ClientId = builder.Configuration["Authentication:Google:ClientId"];
    options.ClientSecret = builder.Configuration["Authentication:Google:ClientSecret"];
    options.CallbackPath = "/signin-google";
});

builder.Services.AddAuthorization();
```

- **Ưu điểm:**

- Giảm bớt gánh nặng quản lý mật khẩu và bảo mật tài khoản.
- Tích hợp nhanh chóng với các nhà cung cấp uy tín.

- **Nhược điểm:**

- Phụ thuộc vào bên thứ ba, nếu nhà cung cấp gặp sự cố có thể ảnh hưởng đến khả năng đăng nhập.
- Cần xử lý logic liên quan đến callback và quản lý thông tin người dùng sau khi đăng nhập.



Thực hành - OpenID Authentication

- Viết 1 ứng dụng ASP.NET CORE MVC hoặc ASP.NET CORE API demo việc sử dụng External Authentication (OAuth/OpenID Connect) để xác thực.



CMC UNIVERSITY

Aspire to Inspire the Digital World



THANK YOU